



Structure and design of multimodal dataset for automatic regex synthesis methods in Roman Urdu

Sadia Tariq¹ · Toqir A. Rana¹

Received: 4 January 2024 / Accepted: 14 July 2024
© The Author(s), under exclusive licence to Springer Nature Switzerland AG 2024

Abstract

Automatic regex synthesis involves generation of regular expressions from user-written natural language descriptions, example strings or both. Daily, countless regex generation queries are posted on online Q&A platforms such as StackOverflow (<https://stackoverflow.com>) and Quora (<https://quora.com>). Existing automatic regex synthesis methods demand concretely designed, multimodal datasets for optimal performance. Unfortunately, publicly available datasets even for resource-rich languages like English are often model-specific and incomplete, potentially hindering the efficiency and accurateness of regex synthesis methods. This issue is worsened for resource-poor languages such as Standard Urdu and Roman Urdu. In this paper, we present a novel, benchmark Roman Urdu dataset with 900 words and a novel Roman Urdu lexicon of 4225 words, annotated and labeled, to address the unmet needs of regex synthesis methods. Equipping these methods with a proficient dataset can lead to more fruitful regex generation.

Keywords Multimodal dataset · Auto regex synthesis · Dataset for Roman Urdu auto regex synthesis · Roman Urdu dataset · Query to regex · Strings to regex

1 Introduction

Urdu, ranked as the 21st most spoken language globally, faces challenges due to a lack of resources for its natural language processing (NLP). Despite being morphologically rich, Urdu encounters issues of resource scarcity, especially in terms of datasets and pre-trained models. The use of Roman-scripted Urdu has surged in popularity, making it the third most widely used language globally [31]. This informal variant of Urdu has become a favored means of communication in the virtual realm, particularly among South Asians.

The ubiquity of user-generated text introduces inevitable challenges such as unintentional misspellings, the incorporation of slang, shortcuts, and abbreviations. This phenomenon is not exclusive to Urdu; even standard languages like English struggle with these issues. Informal languages lack a standardized lexicon which magnifies these challenges. One such

example of spell variants is given in Table 1, wherein we can also witness the resemblance between English and Roman Urdu (using same set of alphabets):

The primary objective of text processing tasks, sentiment classification and sentiment analysis (SA), is to enhance the accuracy of text classifier models. This is achieved through two methods: (1) fine-tuning the model itself or (2) preprocessing the dataset. While model-oriented SA offers inherent understanding of variability and excellent results, they often demand extensive resources and rely on complex architectures. This leads to potential and significant challenges in terms of computational efficiency and complexity [25]. Most recent work includes energy aware resource allocation using CNN and GRU given in Ref. [13], attention-based CNN coupled with transfer learning [34] and combining deep neural network with ensemble learning in Ref. [26]. Model-driven SA can be boosted by the choice and quality of preprocessing carried out prior to classification as [12] showcased by data augmentation for emotion recognition.

Lexical normalization emerges as a pivotal mechanism for dataset pre-processing by reducing text to a canonical form. This process eliminates the coexistence of multiple spell variants of the same word, replacing them with a single correct lexical entry derived from the text. The absence

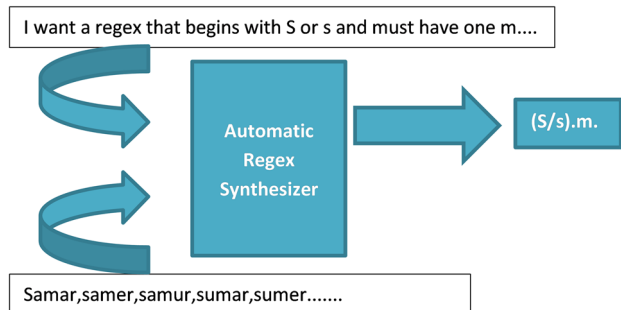
✉ Sadia Tariq
sadia.tariq.shah@gmail.com

✉ Toqir A. Rana
toqirr@gmail.com

¹ Department of CS & IT, The University of Lahore, Lahore, Pakistan

Table 1 Example of spelling mistakes

Language	Correct word	Spell variants
English	weight	wait, w8, wt, wate, etc.
Roman Urdu	bohat	bht, bahot, bhot, buhat, buhut, etc.

**Fig. 1** Automatic regex synthesizer

of tools and resources for standardization in Roman Urdu necessitates a swift and precise mechanism to identify and correct misspelled words.

Regular expressions are a powerful tool widely applicable to text extraction tasks. When crafted cleverly, a single regex is capable to find and rectify hundreds of spell variations of a word in one sweep. The synthesis of automatic regex proves to be a standout method for identifying spelling errors, excelling in both speed and efficiency.

In essence, the challenges encountered by Roman Urdu, the significance of lexical normalization and the instrumental role of regex in handling spelling variations are effectively established. Consider the example given below where one regex will be able to make 52 corrections as shown in Table 2:

Automatic regex synthesis is the process of generating regular expressions accepting text descriptions or from string example or both as input. Therefore, it is a multimodal task in its nature. We display an automatic regex synthesizer in Fig. 1.

Several specialized Roman Urdu datasets for sentiment analysis, containing user-generated reviews and comments can be found online at repositories such as Kaggle and KDNuggets. Examples include RUSAD (Roman Urdu Dataset), RUSA-19, and UCL's dataset, typically featuring two columns labeled as comment text and comment sentiment, respectively. However, these datasets are not directly applicable for regex generation. To synthesize regexes from publicly available dataset, they must first be incorporated into the dataset.

Learning-based automatic regex synthesis is impossible if ground truth regex(actual regex) are not present within the dataset for the annotated lines or words. Alongside the regex, there is a need, for example, strings, both positive and negative, to guide the regex production process accurately.

Therefore, the imperative task is to construct a customized, multimodal dataset that accommodates every requirement of all kinds of regex synthesis methods. By definition the term multimodal means having multiple modes (textual, visual, audio data) in dataset as given in [8]. However, for auto regex models, it symbolizes synthesis of regex from NL descriptions, example strings or both.

1.1 Research motivation

This research is motivated on the following grounds:

1. Recent studies have extensively explored SA in formal languages, particularly resource-enriched like English and Chinese. Urdu, in contrast, is considered resource poor. However, Roman Urdu has gained significant traction in the cyber world, becoming a top choice for communication among South Asians, with an estimated 66 million users globally. Additionally, approximately 697 million users for Roman Urdu have been recorded [31].
2. Regex find applications in program script synthesis, machine translation, and web applications like input validations [41]. These are also utilized in Google Analytics for URL matching and in popular editors such as Sublime, Notepad++, Brackets, Google Docs, and Microsoft Word. The widespread use of regex is evident from the numerous queries related to regex generation on platforms like StackOverflow and Quora.
3. Despite their popularity, manually writing regex has been a laborious and error-prone task even for experienced programmers. Users often view the generation process as a black box, inputting natural language descriptions or example strings and receiving a regex as output.
4. Automatic regex synthesis is well studied for English and various methods exist to serve the purpose. However, we have observed a criminal lack of survey or review articles based on their comparative analysis.
5. With auto regex synthesis, customized regex synthesis for most frequently appearing and most likely misspelled words in dataset will be achievable. It will lead to normalization of dataset which in turn supplements classifier's performance.

Table 2 Example of regex-based lexical normalization

Correct word	Incorrect variations (occurrences)	Number of replacements	Pattern/Regex
Samar	Samar (12) Samer (10) Sumar (10) Sumer (10) Samur (10)	52	(S/s).m.r

1.2 Research objectives

The objectives of this research are as follows:

- To highlight significance of automatic regex synthesis for Roman Urdu
- To build a novel Roman Urdu dataset that contain Natural Language (NL) descriptions, example strings and ground truth regex
- To build a Roman Urdu lexicon containing positive and negative words, labeled and ready to serve all modes of auto regex synthesis

1.3 Research contributions

Our contributions in this paper are summarized as follows:

1. Discuss various methods and modes for auto regex synthesis
2. Critically analyze various datasets available for the said task emphasizing their model-crippling issues
3. Prepare and label a novel corpus (900 words) and a novel lexicon (4225 words) tailored to boost model's accuracy
4. Emphasize usage of regex as language production tools

1.4 Methods

This research primarily highlights significance of automatic regex synthesis in Roman Urdu text processing and critically analyzes datasets available for the said task. It is an under-utilized area of language study with scarce studies for even Standard English language. Since neither such models nor such task specific datasets for resource poor Roman Urdu language are available, we follow footsteps for English due to their resemblance. We prepared a novel corpus and a novel lexicon to provide for automatic regex synthesis models for Roman Urdu. Section 2 contains category wise, brief literature survey of existing methods and datasets for regex synthesis for English. In Sect. 3, we highlight the limitations of existing methods and gaps in available datasets. Section 4

explains the structure and design of proposed multimodal dataset. Section 5 elaborates the dataset preparation process. In Sect. 6, we proposed a framework to use proposed dataset for solving a downstream task empirically. Section 7 concludes the research work along with recommendations for future work.

2 Brief literature survey of regex synthesis methods and datasets

Despite availability of models and methods for English language, a lack of comprehensive background studies has been observed. This section is about introduction and analysis of existing methods and datasets for regex synthesis task available in literature for English Language.

2.1 Regex synthesis methods: categories and their examples

There are three modes of regex synthesis named: (1) purely NLP based, (2) purely string examples based and (3) hybrid method. NL descriptions are confined by imprecision and ambiguity [19]. Purely NLP-based methods generate regex from NL text snippet written by user. Purely string example-based methods use example strings (both positive and negative) to infer a regex. Hybrid methods utilize both NL text clips and string examples for the said task. By producing a dataset that serves all three kinds of methods, our contributions become twofold.

These methods can also be categorized another way into three broader categories, namely (1) rule-based methods (2) grammar-based methods and (3) learning-based methods. Further granular subdivisions are as follows.

1. *Pedagogical based*: Pedagogical regex synthesis method uses positive and negative examples that accept all positive strings and reject all negative ones. Alpha-Regex presented by Lee et al. [18] uses enumeration-based algorithms. Regexgenerator++ by Bartoli et al. [2] performed regex synthesis solely based on examples just like inferring an acceptor for a regular language based on positive and negative sample strings, representing strings described by the language and of strings not described by the language,

respectively. Kim et al. [14, 15] explore problem of synthesizing regex from a set of positive and negative strings. As an improvement in best-first enumeration of regex, they define a new normal form of regex called the concise normal form. It allows significant reduction in the search space by trimming the regex that are not in the normal form.

2. *Active learning & GP based*: In active learning user annotates only one desired regex extraction and then merely answers extraction queries generated by the system. GP regex golf by Bartoli et al. [1] is genetic programming (GP)-based method to generate shortest regex given a set of sentences both positive and negative.

Bartoli et al. [3] used regex-based entity extraction from unstructured data based on active learning and GP whose ultimate goal is to avoid costly and error-prone manual annotation effort.

3. *Neural Network(NN) based*: These methods utilize the power of neural networks and learning for regex production. *KB13* model by Kushman et al. [17] learn to generate regex from natural language, using a training set of natural language and regular expression pairs. It utilizes domain specific pieces which compute the semantic likeness between two regexes. But it cannot be used for other formal representations where such equality calculations are not applicable. Famous domain agnostic neural model named *DeepRegex* has been presented by Locascio et al. [22]. It uses large corpus of line level regex and strings pairs for regex synthesis problem through neural machine translation. A new Dataset named NL-RX-Synth with 10,000 NL-Regex pairs was created automatically. A generate-and-paraphrase procedure [40] was utilized to create NL-TURK dataset where NL queries were later paraphrased by human participants. This popular model is adaption ready for Roman Urdu due to character level translation very similar to English.

Chen et al. [4] introduce semantic regex, a broader form of regex that enables both syntactic and semantic analysis of textual data. A learning algorithm was implemented in a tool named *Smore* to efficiently synthesize semantic regexes from limited positive and negative examples through a combination of neural sketch generation and compositional type-direction.

4. *Probabilistic Regular Grammar (PRG) based*: These methods infer regex by incrementally growing a grammar using positive and negative string examples and then convert them into regex. By using *Bayesian approach* Ouyang et al. [27] assign high prior to shorter regexes and higher likelihood to good regexes (that explain the example string well) and perform approximate inference. In this way, the issue of uninformative nature of user-generated strings is resolved.

5. *Sematic Similarity based*: Semantic similarity, defined as semantic relatedness in Ref. [35], is an active field of research. Semantic-based approach believes in the similarity

of two model outputs based on the similarity of their corresponding Deterministic Finite Automata (DFA) rather than using string examples. The model proposed by Zhong et al. [44] is based on semantic correctness using DFA equivalence as an estimate and is named as *SemRegex*.

In *SoftRegex*, Park et al. [28] used regex similarity problem from its baseline predecessor, *SemRegex*. They proposed *Eq_reg* model that computes similarity using deep NNs. Rebele et al. [32] argue that hand-crafted regexes may fail to match all the intended words. They proposed to generalize a given regex so that it matches also a set of missing words. By finding an approximate match between the missing words and the regex, and by adding disjunctions for the unmatched parts appropriately, they improved the precision and recall of the regex. In addition, they generated much shorter regexes than baselines and competitors on various datasets.

6. *Programming by Example (PBE) based*: A model that synthesizes rules from examples like input–output pairs for the end user who does not know how to program rules or has clear examples in mind but wishes to outsource the burden of writing the rules. This is known as programming by example. Ye et al. [42] perform synthesis of regex by jointly employing NL and examples proposing new framework named *Regel*. *Regel* parses the English description into tiered sketches for a PBE engine.

Zhang et al. [43] addresses the persistent challenge of ambiguity in user-provided examples within PBE. Introducing an interaction model, they offer two types of augmentations to user-given examples in regexes: user-oriented semantic augmentation and synthesizer-oriented data augmentation. However, a user study demonstrates that interacting with augmented examples significantly enhances task completion rates, reduces time, and increases users' confidence.

Pertseva et al. [29] argue regexes are a popular target for PBE systems, which seek to learn regexes from user-provided examples. One possible solution is to input the two examples into a regex synthesizer and get a pattern-matching expression as output.

Vaithilingam et al. [39] argue that user-specified examples often introduce ambiguity and to address this challenge frame program synthesis tasks as pragmatic communication. Extending this to regexes and rigorous examination of the usability of pragmatic communication, they found that end-users could more successfully convey a target regex using positive examples. Chida et al. [6] highlight that recent PBE methods for regex synthesis only cater to membership testing, leaving extraction as an open problem. Their method is implemented in a tool called *R3* (Repairing Regex for extraction) and allows substantial reduction in the search space through novel pruning techniques.

Some similar program synthesis examples include generation of SQL queries [38], spreadsheet formulas [9] and java

expressions [10] and bash formulas [21] from NL descriptions.

7. Synthesis & Repair based: These methods synthesize regex in stepwise manner and repair them until they become acceptable according to user given criteria. If not crafted sensibly, regex may lead to denial of service due to their super-linear nature and exponential consumption of resources. In Ref. [7], an empirical study of *ReDoS* (Regex-related Denial of Service attack) discovers the occurrence of super-linear regexes and concept of anti-patterns (problematic and sub-optimal patterns). These anti-patterns are essential but not sufficient to understand super-linear behavior.

Shen et al. [37] propose *ReScue* to automatically generate ReDoS strings and highlight vulnerabilities of given regexes.

FlashRegex presented by Li et al. [20] has been tested on five state of the art tools. Comparatively, older tools generated 394 Re-DOS vulnerable regexes within few seconds, whereas it generated no such potentially dangerous regex for at least 5 seconds. This performance is even better than human expert crafted regex, plus its user friendly, general and automatic.

Kim et al. [14, 15] propose *SplitRegex*, a search and repair strategy-based efficient regex generation, divide and conquer framework. The fundamental goal is to avoid inherent p-space complexity of regex synthesis.

McClurg et al. [24] automated the removal of inefficiencies in regexes by introducing a new approach called rewrite-guided synthesis (*ReGiS*), which combines Syntax-guided synthesis (*SyGuS*) with equality saturation-based rewriting. This unique interplay overcomes cost and saturation challenges, resulting in an efficient and scalable framework for expression optimization.

8. Divide & Conquer based: Splitting a problem into smaller sub-problems is a classical approach, which aims to find partial solutions and then combines them to make one global solution. This is called divide and conquer-based synthesis. Rahmani et al. [30] jointly employ pre-trained model (PTM) to generate candidate programs from NL specifications and component-based system (CBS) to rank them from best match to worst match with the examples. It has been evaluated on two domains: regexes and cascaded style sheet (CSS) selectors. Kim et al. [16] present an effective regex synthesis framework that constructs sub-regexes from segmented positive substrings and combines them to form the ultimate regex. Using pre-generated sub-regexes during negative sample analysis ensures consistency with all negative strings, making a divided-and-conquer approach that significantly enhances regex synthesis accuracy across four benchmark datasets compared to previous methods.

9. Multimodal: These methods generate regex from both NL descriptions and example strings. *TransRegex* given by Li et al. [19] has been evaluated on 10 relevant tools and 3

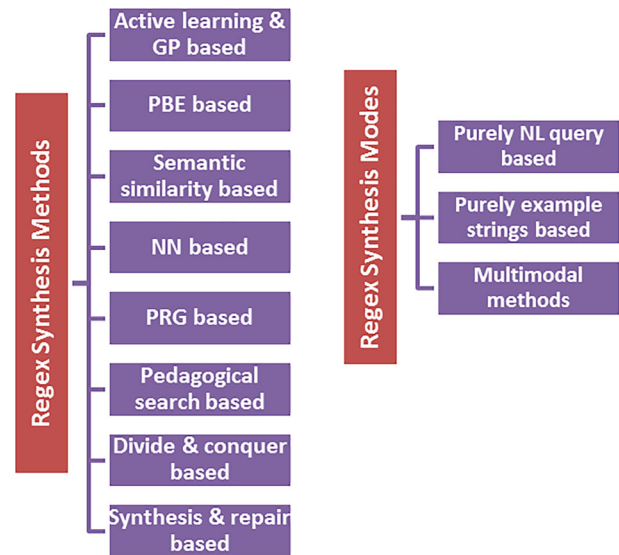


Fig. 2 Regex synthesis models and modes

benchmark datasets. As compared to most recent multimodal methods, its accuracy is 10–30 percent higher.

Figure 2 reveals a hierarchical summary of the aforementioned models and modes.

Neural network-based model *DeepRegex* offers more than 3 times accuracy improvement over its baselines (BoW-NN and Semantic Unify). It can easily be declared the top notch model for auto regex synthesis. *SemRegex/SoftRegex* are even better than *DeepRegex*, giving 2–12.6 percent accurate results in their best case on KB13 dataset. In terms of speed, semantic similarity-based methods like *SoftRegex* give good results by speedily producing regex in less than 5 s. Also, PBE and synthesis plus repair-oriented models are researchers' favorite for further productive experimentation.

2.2 Popular datasets

A list of popular datasets utilized in the studies mentioned above is given below:

1. KB13
2. NL-Turk
3. NL-RX-Synth
4. Student Assignments
5. M-Turk

KB13 model and dataset presented in [17] have been found to have repetitions up to 45%, while NL-RX has up to 97% unique regexes.

KB13 acquires high performance on baseline neural nets with 29% improvement in accuracy. It outperforms exact string-based unification by over 30%, example-based

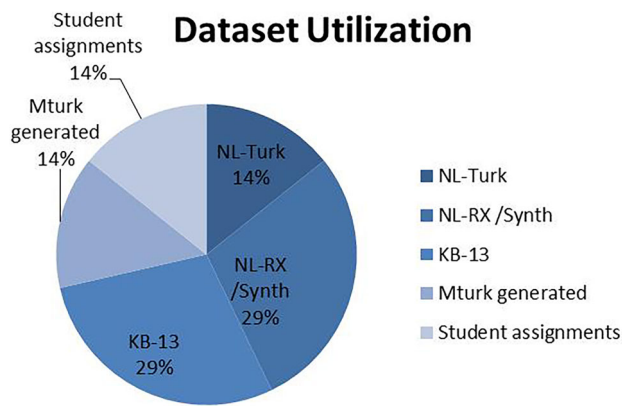


Fig. 3 Dataset Utilization

semantic-unification by over 13% and smart heuristic-unification procedure by 9%.

Zhong et al. [44] performed real versus synthetic dataset debate highlighting the very low effectiveness of learning-based regex generation methods for real datasets. The utilization of popular datasets is given in Fig. 3.

From Fig. 2, the following information is obvious:

- KB13 is the most popular realistic dataset used for regex synthesis
- NL-RX-Synth is the most popular synthetic dataset for the purpose

The recent datasets offer synthetic examples, whereas the ideal dataset would be a balanced mixture of real and synthetic examples. Models need be improved to work better for realistic datasets and datasets need be improved to cover up for models' inabilities.

3 Limitations of existing methods and datasets

Both the models and the datasets have their flaws for the chosen task, automatic regex generation. Let us have a bird eye look at them in one by one manner and begin with issues in regex synthesis methods.

Most of the recent work falls in the category of either improving text classification performance by updating model architecture and parameter settings and rarely by preprocessing the lexicon. Spell correction alone is insufficient for achieving lexical normalization goals completely as there are many other challenges. String matching based on edit distance becomes cumbersome to compute for longer words. We need a method to identify all possible variations of a word. Nonetheless, we seldom see any effort to achieve performance speedup by employing regex.

Regex is used to find matches for patterns within data strings. Ideally, if somehow we identify the right number of misspelled words that need a correction, regex is perfectly able to generate a gold standard benchmark for resource poor languages such as Roman Urdu. This is where an automatic regex generator is required. Next, we highlight issues in existing datasets.

The datasets can be either realistic or synthetic in nature. Realistic datasets contain real-world instances only and synthetic ones contain auto-generated instances. Existing datasets turn out to have following issues:

1. Synthetic datasets are *simple and small sized* with lesser variety of language, as compared to real-world queries posted to Stack Overflow and similar online platforms.
2. Recent synthetic datasets have *fewer than needed unique pairs*. KB-13 [17] has 814 pairs with fewer unique pairs though NL-RX (Locascio et al. [22]) and its modification who have 10,000 pairs.
3. NL-Turk, using NL-RX (Locascio et al. [22]) and generate and paraphrase procedure from [40] also have *non-diverse, simple and short* paraphrased descriptions. Other than paraphrasing, improvement of weak queries is also essential and discussed in Ref. [11].
4. Random sampling causes *simpler regexes*.
5. StackOverflow dataset [42] on the other hand is realistic and complex but contain only 62 NL-regex pairs which makes it *unsuitable for large scale training*.

For an automatic regex generator to perform faster and accurate lexical normalization, a novel, multimodal Roman Urdu dataset is obligatory. It is expected to be well sized, reasonably complex and contain a good mix of synthetic and realistic queries as well as regex. Next, we consider two standard datasets and attempt to eliminate the issues that cripple regex synthesis models.

4 Multimodal dataset's in literature: benchmarks

The hardest challenge for an informal language is the absence of a universally accepted, standard dictionary, and hence, no single dataset can be considered gold standard. *Our key idea is to construct a benchmark dataset for Roman Urdu on the same pattern as applied to stylized English language datasets used for regex synthesis*. Somehow we should unite user-written NL scripts and their corresponding ground truth regex (most likely regex) in pairs, and for this purpose, we examine three already existing datasets.

Table 3 Regexes for extraction of weight values from annotated text snippets

Text specification	Sample regex
Most recent visit: Oct 7, 2005, actinic changed, face, cyro ear/forearm, plan: surg tx in about one mo, b/p 155/75 wt 187 2005 visits: multiple for lesions, 7/2005 b/p 145/75, 5/2005: surg txs: 'suspect, squam cell CA' b/p 130/55	(?i)wt\s{1,2}(\d +)\s{1,2}
Polyp was excised and found to be benign. He should have follow-up sigmoid after 5 years. BP = 133/88 P = 68/min Wt = 154 lbs Good carotid upstroke without bruits, no oral lesions, ear canals have some wax but TM's are clear	(?i)wt\s{1,2}[pPunc]\s{1,2}(\d +)\s{1,2}
No need for pain med. He has gained about 35 lbs. since discharged from active duty and has lost weight already as he is starting to exercise more and count calories	N/A

4.1 REDEx

Redd et al. [33] used a new approach using regex discovery for extraction (*REDEx*) has been demonstrated. With a supervised machine learning algorithm that automatically generates regex to extract measurements with reduced effort. Compared to support vector machine (SVM) for specific task of body weight extraction from EHRs (electronic health records), the *REDEx* model consistently outperformed SVM. Instances in the dataset contain pairs of snippets retrieved using the keywords and the regexes resulting from *REDEx* training as shown in Table 3.

Mostly new models are tested on synthetic datasets but fall flat on real-world dataset. Much lower effectiveness on real-world datasets is being achieved as compared to synthetic datasets for famous *DeepRegex* model. As a bold attempt, Zhong et al. [45] used synthetic dataset for training and tested on real one. Real-world dataset named *RegexLib*¹ has been obtained from online library containing 3619 pairs extracted from search interface of the library. Table 4 shows word count of regexes produced for these datasets.

Similarly, Table 5 compares length of regexes in these datasets.

Two main distinguishing elements of the realistic dataset are: (1) Complexity of regex and (2) Complexity of NL snippet.

In *RegexLib*, average regex length greater than 40 is common but for other two synthetic datasets, average regex length is between 10 and 40 characters. For example, the character '?' is absent in synthetic but present in real dataset. Real-world dataset has been found to possess more complex NL descriptions with word count lesser than 20 whereas for synthetic approach word count lesser than 20.

DeepRegex is 0.5% effective on realistic dataset, whereas 80.9% on synthetic which is a sharp contrast. *DeepRegex* has three major issues:

1. NL variations in dataset must be removed before training the model
2. Specifications which contain numerical ranges must be handled
3. Also, real dataset is sparse, whereas synthetic dataset is dense due to annotation and paraphrasing. Real-world dataset contains more distinct words than synthetic ones

Another major challenge is to label the massive collected real-world data. String examples have proven to be good for disambiguation of NL specifications thus helps differentiating candidate regexes. In [17], the authors believe that a given regex can be written in many semantically equivalent forms, and they facilitated NL to regex translation by finding a form which more directly corresponds to the NL. *The instances in dataset contain pairs of text description and associated regex.*

In addition, Satti et al. [36] present a novel pipeline using transfer learning, clinical concept dictionaries, and pattern matching to provide an end-to-end solution for identifying attributes and extracting their values from natural clinical text.

4.2 StructuredRegex

Chen et al. [5] suggested a dataset named *StructuredRegex (STReg)* for regex synthesis. It is unique in three aspects: (1) Regexes are generated using probabilistic grammar with pre-trained macros gathered from real world StackOverflow posts, (2) Obtain diverse NL descriptions rather than paraphrasing from synthetic language and (3) Each regex is provided with multiple strings both positive and negative based on ground truth regex.

It is a large scale, realistic dataset with triplet information: (1) NL snippets, (2) Example strings and (3) Regexes. Therefore, it is truly multimodal by nature. The statistics of explored datasets within this work are displayed in Table 6.

Few complex and realistic StackOverflow examples are shown in Table 7. The process used for dataset collection is depicted in Fig. 4.

¹ <https://regexlib.com/>

Table 4 Word count statistics

Dataset	Maximum	Average	Median	SD
KB-13	21	7.1	7	2.8
NL-RX	28	10.6	10	3.3
RegexLib	968	36.4	23	45.5

Table 5 Length of regex statistics

Dataset	Maximum	Average	Median	SD
KB-13	79	19	18	11.9
NL-RX	54	26	25	7.4
RegexLib	685	58.5	53	44.2

Table 6 Statistics of the datasets explored

Dataset	KB13	NL-RX-Turk	STReG	SO
Size	824	10,000	3520	62
#.unique words	207	557	873	301
Avg. NL length	8	12	33	25
Avg. reg size	5	5	15	13
Avg. reg depth	2.5	2.3	6.0	4.0

StructuredRegex has 3520 instances. They used visual symbols and glyphs to draw given regex with minimal textual hint. Testing with state-of-the-art techniques, 84% performance was achieved on NL-Turk. *StructuredRegex*'s performance over same techniques was 37%.

4.3 DeepRegex

Famous domain agnostic neural model named Deep-Regex has been presented in Locascio et al. [22]. DeepRegex is long short-term memory (LSTM)-based sequence-to-sequence neural network with two stages generate and paraphrase procedure. It used large corpus of regex and strings pairs for regex synthesis problem. It subsequently employed a generate-and-paraphrase procedure to create NL-TURK dataset. In generate phase, small manually crafted grammar was used to translate regex into NL. During paraphrase phase, rigid descriptions were converted to descriptions that are more natural.

The novel Dataset thus created was named NL-RX with 10,000 NL-Regex pairs. Performance gain of 19.6% has been

achieved. Further insight into the working of the model will be given by an example NL description, its corresponding paraphrased version and regex given below:

Synthetic NL snippet: lines not words with starting with a capital letter
Paraphrased NL snippet: lines that do not contain words that begin with a capital letter
Regex: $\sim (nb([A-Z])(.*)nb)$

4.4 The structure and design of proposed dataset

From these benchmark datasets applicable to smart regex generators, we acquired guidelines to build multimodal dataset. There will be three instances containing text clips, string examples and target regexes. The dataset has to be realistic with large sized, unique words and complex regexes for the models to work better. We intend to build a dataset as exhibited in Table 8 for English.

Table 8 offers a glimpse of the proposed dataset built for Roman Urdu by following footsteps of Table 9.

Table 7 Realistic examples

Text snippet	Resultant regex
I need to validate the next pattern: starts with "C0" and finish with 4 digits exactly	$and(startwith(< C0 >),endwith(rep(< num > ,4)))$
I need regex for: one or two digits then"." and one or two digits	$concat(reprange(< num > ,1,2),concat(< . > ,reprange(< num > ,1,2)))$

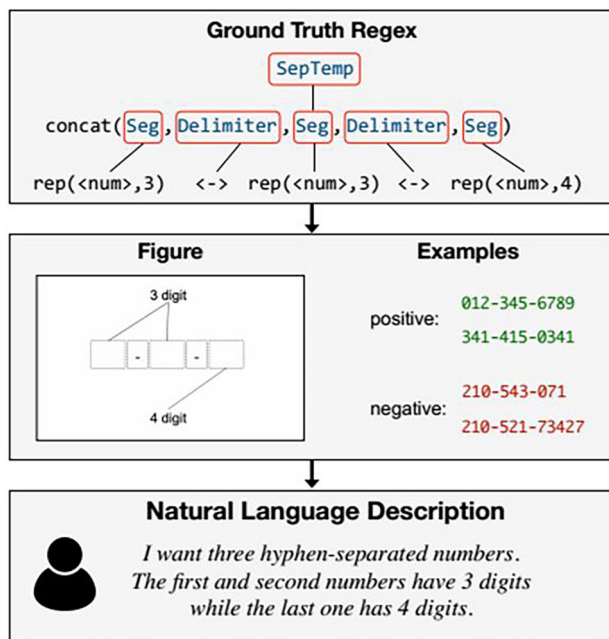


Fig. 4 Dataset collection process of *StructuredRegex*

5 Dataset preparation process

Based on our observations from previous section, we created a new multimodal dataset, which is based on famous publicly available Roman Urdu datasets: (1) RUSA-19 and (2) UCI repository's dataset.

We filtered out the comments with positive sentiment, as they were lesser in number. For example, the filtered RUSA-19 dataset contained 3780 instances, out of which 220 words with many spell variants were chosen/annotated.

Each instance was manually labeled in the dataset with a regex that covers all of the spell variants in that instance.

Finally, after the annotation and labeling task is complete, NL queries were written for them. Similar process was repeated for negative comments in the dataset. A systematic flowchart of the dataset preparation process is shown in Fig. 5.

A screenshot of the newly constructed Reg-Gen datasets along with that of NL-RX-Synth is shown in Figs. 6 and 7, respectively. Clearly, the former contains word level regex for Roman Urdu, whereas the later contains line level regex for Standard English.

5.1 Dataset description

Data Sources: We chose two publically available datasets, RUSA-19 and UCL's dataset, for the creation of our dataset. The UCI repositories' Roman Urdu dataset, which contains 20,229 instances and 2 attributes arranged in CSV format. The first column contains subscriber/user written comments collected from various platforms and the second column contains the sentiment assigned to them. The sentiments have been classified into three broader categories: positive, negative and neutral.

The RUSA-19 Corpus comprises 10,021 Roman Urdu sentences. Each sentence falls into one of three classes: positive (denoted by 1); negative (denoted by 2); and neutral (denoted by 0) [23]. Among its massive count of 10,021 instances, 3290 comments have been labeled as neutral.

Along with these standard datasets, we have put together two lexicons of positive and negative Urdu words written in roman script. These contain around 4225 words and we labeled most of them using their common spelling variations collected from various resources.

Table 8 Proposed Dataset for English

Text clip	String examples	Regex/Ground truth regex
I want a regex that accepts all words starting with 'a,' followed by double 'p' and contain one or many characters afterwards	apple, approve, appease etc	app*
Your approval is essential	approve, approval, appease, apple, appoint etc	app*

Table 9 Proposed Dataset for Roman Urdu

Text clip	String examples	Regex/Ground truth regex
I want a regex that targets all words that begin with 'b' and end at 't' and must have one 'h' in between	boat, boot, behest, bohat, bht etc	b(?:oh[aeou]tlht)
Ap bht achay hain	bht, bohot, bohat etc	b(?:oh[aeou]tlht)

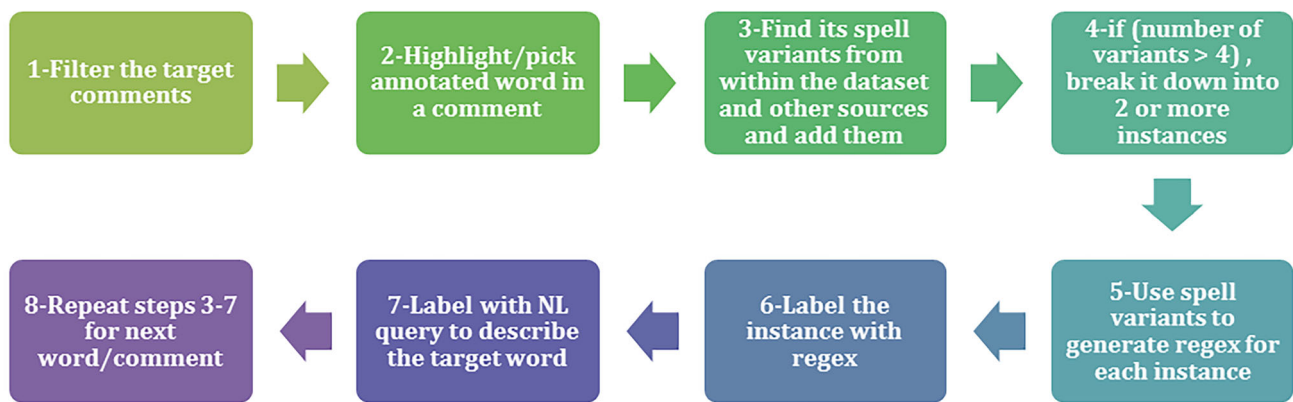


Fig. 5 Dataset preparation process

File	Home	Insert	Page Layout	Formulas	Data	Review	View	
J9								
	A	B	C	D	E	F	G	H
1	NL clip	Regex	Example strings					
2	words that begin with'h' and end at a,y,e and m	h(?:a(?:i?y i) [ei]y)	hay	hy	haiy	hi	hai	he
3	words that begin with d and end at a and must c	d(?:[ei])?y [ei]a	diya	dia	dya	deya	dea	
4	words that begin with r and contain kh in betwe	r(?:ak(?:he(?:in y) ein) khen)	rakhey	rakein	rakhein	rkhen		
5	words that begin with m and end at y n or l, con	m(?:ain e(?:in y) ai i)	mey	mein	main	mi	mai	
6	words that begin with sh and contain one r	sh(?:ur(?:oo u) ro?u)	shuru	shru	shrou	shuroo		
7	words that begin with a contain g in middle and	ag(?:ga (?[ae])?)r	ager	agr	aggar	agar		
8	words that begin with k and end at n ,must cont	k(?:ar(?:ei?n yen) erein)	karen	karein	karyen	kerein		
9	words that begin with k and end at e or y ,must	k(?:ar(?:ay [ey]) ray)	karay	kary	kare	kray		
10	words that begin with li or ly end at e or a	l(?:i(?:[?y]ea e) ye ye)	lye	liye	lie	liyea		
11	words that begin with le or li end at y or h	l(?:e(?:yeh a[hy]) iey)	leay	leyeh	leah	liey		
12	words that begin with d and end at n ,may conta	d(?:[ae][iy])? ia)n	dn	dain	dayn	dein	dian	deyn
13	words that begin with t and end at h or a must c	t(?:ar(?:[ha]h h) erha ra)	tarah	tarh	tarha	tra	terha	
14	words that begin with tar and end at n must cor	tarhaa?n	tarhan	tarhaan				
15	words that begin with k and must contain rt in b	k(?:art(?:ai [ey]) rtay)	karte	krtay	karty	kartai		
16	words that begin with chah end at y a e or n	chah(?:i(?:[?y]en ay) eye ya)	chahiyen	chahiay	chahya	chaheye		
17	words that begin with ch and may contain anot	ch(?:a(?:hiy e) a?ye)	chye	chahiy	chaye	chae		
18	words that begin with d, must contain u and o i	do[ou]?sr[uo]u?n?	dosrun	doosro	dosroun	dousroun		
19	words that begin with ba,bh and end at a,e,i	b(?:ah[ei] h(?:a[ei] ia))	bhae	bhai	bahi	bhia	bahe	
20	words that begin with ba,bh and end at l or a m	b(?:a(?:hiya i) hayya)	bai	bhayya	bahiya			
21	words that begin with b must contain one h and	bh[iey]	bhi	bhe	bhy			
22	oneor two letter words that begin with b	be?	b	be				
23	words that begin with h and end at m, containi	h[ua]?m	hum	hm	ham			
24	words that begin with h and end at a,i or y,cont	h(?:[?ai]?y ai?)	ha	hai	haiy	hy		
25	words that begin with m and end at t, may cont	m[at]?t?	mt	matt	mat	mtt		
26	words that begin with h and end at n, containin	h(?:[ae]i y?)n	hain	hein	hvn	hn		

Fig. 6 Reg-Gen lexicon's screenshot

Data Collection Process: The manually labeled regexes were generated by using online tools, which take example strings as input and generate regex as output.

Data Format: The format of the dataset is CSV file. Two separate CSV files exist, one for each parent dataset.

Data Size: The RUSA-19-based dataset contains 485 instances for 365 unique words. The UCI based dataset contains 415 instances for 325 unique words. This difference is mainly attributed to more spelling variations per word found in RUSA-19 than in UCI.

5.2 Data attributes

The following attributes were already present in the publicly available datasets:

Comments: Natural language text in Roman Urdu.

Sentiment: Positive (1), Negative(2), Neutral(0).

The following new attributes have been added to ensure suitability of the dataset as input into regex synthesis models:

Regex: The regex that represents the given example string and is manually labeled into the dataset.

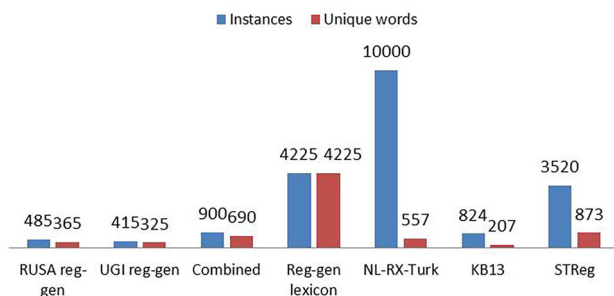
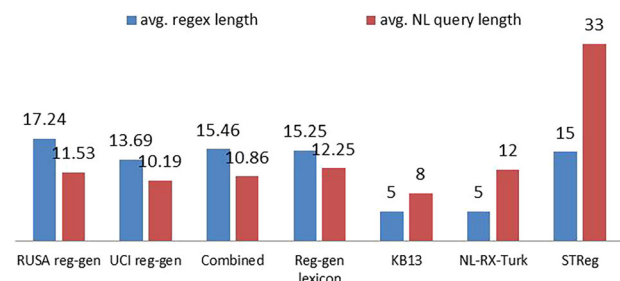
Annotated Word: A words from within the comment text, such as, “dekhte.”

Fig. 7 NL-RX-Synth's screenshot

C20		f _x
A		B
nl_clip		regex
lines with words with a capital letter and a vowel		\b([A-Z])&([AEIOUaeiou])\b
lines ending with a character before the string 'dog'		((.*)(.))*.(dog).*
lines containing a letter or a character		.*([A-Za-z]) (.)*
lines with a number before a letter and a lower-case letter		(([0-9]).*([A-Za-z])&([a-z])).*
lines containing the string 'dog', 2 or more times		.*(dog){2}.*
lines with the string 'dog' or the string 'truck', 5 or more times		((dog) (truck)){5,}
lines ending with a letter followed by a number		(.*)((A-Za-z).*[0-9]).*
lines with the string 'dog' and the string 'truck' before a number		((dog)&(truck)).*([0-9]).*
lines with words with a number or ending with a character		\b([0-9]) ((.*)(.))*\b
lines with a vowel, zero or more times before a lower-case letter and a capital letter		((([AEIOUaeiou]).*([a-z])&([A-Z])).*
lines having either a character, the string 'dog', or a number before the string 'truck'		((.*)(dog) ([0-9])).*(truck).*
lines starting with a vowel at least once or the string 'dog'		((([AEIOUaeiou])+) (dog)).*
lines with words with either a letter, a capital letter, or a lower-case letter before the string 'truck'		((b([A-Za-z]) ([A-Z]) ([a-z])\b)+
lines with the string 'dog' before the string 'truck' or not the string 'ring'		((dog).*(truck).*) ((ring))
lines not words with starting with a capital letter		~\b([A-Z])(.)*\b
lines with words with a number and a capital letter before the string 'dog'		\b([0-9])&([A-Z]).*(dog).*\b
lines containing either a number, a lower-case letter, or the string 'dog'		.*([0-9]) ([a-z]) (dog).*
lines containing only a number		[0-9]
lines containing ending with a lower-case letter		.*([a-z]).*
lines with the string 'dog' before the string 'truck' and a letter		(dog).*((truck)&([A-Za-z])).*

Table 10 Statistics of the datasets

Dataset	Word count	Avg. Regex length	Avg. NL query length	No. of unique words
RUSA reg-gen	485	17.24	11.53	365
UCI reg-gen	415	13.69	10.19	325
Combined	900	15.46	10.86	690
Reg-gen Lexcion	4225	15.25	12.25	4200

Size of datasets**Fig. 8** Size of datasets**Label length of datasets****Fig. 9** Label length of datasets

Example Strings: Various possible spellings of the annotated words, mostly from within the dataset but can be from other sources such as internet or other public datasets. The dataset statistics are shared exclusively in Table 10 as well as graphically illustrated in Figs. 8 and 9, respectively.

It is clearly notable that line level regex-based NL-RX contains 10,000 instances but only 557 unique tokens whereas in word level regex-based Reg-gen lexicon almost all words

are unique. In the presence of this much uniqueness, it may even need data augmentation to add essential noise before being fed into any auto regex synthesizer.

All Reg-gen-related datasets have longer and deeper regexes than the already existing ones. Also, their NL query length is shorter than regex length but still comparable to NL-RX dataset. It has been established that all Reg-gen dataset statistics are comparable to their existing equivalents.

5.3 Potential applications

This dataset is now able to augment various automatic regex synthesis models to learn better. In this way, they can generate more precise regexes. The input may be a natural language description or a set of example strings or both. The dataset is a satisfactory reply to the following research questions:

1. How do regex synthesis models benefit from the dataset to improve their efficiency?
2. How does the word level regex generation help in sentiment classification or other text processing tasks?
3. Can a single, unified dataset work wonders for various different kinds of synthesis models?
4. Since the spelling variations are mostly dataset dependent, will they work for other datasets?

5.4 Quality and preprocessing

During the dataset preparation process, new feature named, regex, was created by using existing and newly added attributes. We created it by using raw characters, character classes and common metacharacters used to the purpose and labeled it manually into the dataset. We chose the words with minimum 2,3 spell variations existing within the dataset. We filtered the positive comments only as they were less in number for the annotation task. To assist the learning process and favoring shorter regexes, we broke down a word with more than four variations into multiple instances. The spell variations that looked almost alike were kept intentionally in one row, thus favoring simpler regexes.

5.5 Ethical and social implications

The ethical and social implications of this dataset are significant. It is crucial to address potential issues within the dataset to prevent biased outcomes and ensure fairness of any research it is used in. It is important to highlight that the regex added to the dataset does not claim to capture all possible spelling variations of a word. They solely depend on the quality and quantity of the variations added by various groups of participants. The NL queries also depend on the regex generated from those spell variants. We claim to have ensured correct, double checked labeling to avoid any kind of bias, misuse or abuse.

6 Theoretical framework for downstream task

Reg-gen datasets contain specific, complex, longer and deeper regex patterns. No prior auto regex synthesis model

exists for Roman Urdu. Even preliminary testing on simple model adapted from English will require refinement of regexes and augmentation of dataset. Otherwise model may not generalize well and may over-fit. The NL descriptions in Reg-gen datasets provide a rich source of information for generating simpler, rule-based regex patterns. These patterns, while less complex than the specific and actual regex, can still effectively capture the essential spell variants. Initially, we propose using a seq-to-seq (sequence-to-sequence) learning model. This model can learn from the given NL clips and regex pairs to generate dataset-specific regex and is evaluated using NLP evaluation metrics such as ROUGE and BLEU scores.

An interesting downstream task solved using the proposed dataset will be lexical normalization using regular expressions for SA in Roman Urdu. The theoretical framework involves several key steps:

Intrinsic evaluation methods

We use intrinsic quality metrics to evaluate the dataset, ensuring consistency, completeness, and overall data quality. Visualizations such as charts and tabular data distributions provide a comprehensive understanding of the dataset.

Rule-based regex generation

We suggest to utilize the NL descriptions to create simpler regex patterns for preliminary tests. These patterns serve as the basis for training the seq-to-seq model.

Seq-to-Seq model training

Next, training of the seq-to-seq model using the pairs of NL descriptions and their corresponding regex will take place. The Reg-gen dataset is perfectly able to normalize every RU dataset up to some extent. It can be further generate regex for newer, unseen words customized to the dataset.

Spell correction using generated regex

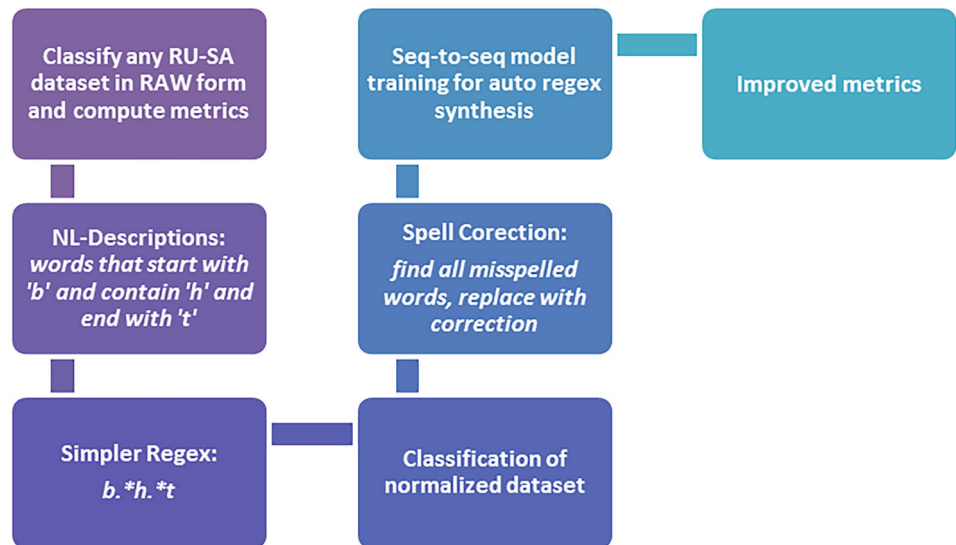
Finally, apply the regex for spell correction via a find and replace strategy. This step preprocesses the dataset, aiming to rectify spelling errors in erroneous dataset and removes inconsistencies.

Improvement in classification outcomes

Preprocessed datasets are expected to yield better performance in downstream classification tasks. The diagram in Fig. 10 illustrates the proposed framework:

7 Conclusions and future recommendations

Roman Urdu is a widely popular language, primarily observed in online user-generated text, comments, and queries. Due to its resource-poor nature, the absence of a universally acknowledged gold standard dictionary, and a flexible word and sentence order format, it poses challenges for the performance of text processing models. Lexical normalization is a task of text pre-processing that enhances dataset quality by standardizing the text. These applications

Fig. 10 Proposed Theoretical Framework

intend to find and replace all incorrect variations of a word with a single, correct lexical entry. Regex proficiently performs this task for massive textual datasets. To generate faster and more accurate regex, there is a need for auto regex synthesis models. Unfortunately, publicly available datasets for both Roman Urdu and English are unsuitable for automatic synthesis of regex.

In response to this gap, we proposed and created a dataset through manual annotation and labeling, derived from an existing dataset. This dataset can be seamlessly input into an automatic regex synthesizer model, enabling the generation of fast and accurate regex for the lexical normalization task. A significant impact of this dataset will be a shift in SA methods toward lexicon improvement rather than model-driven SA, contributing to an excellent computational understanding of the language.

In future, this dataset will be utilized by all types of auto regex generation methods. New models may be proposed or existing models for English can be adapted to work for Roman Urdu. Also, multimodal regex synthesis will be possible generating regex from NL clips, example strings or both. While the Reg-gen Lexicon can serve every Roman Urdu dataset instigating some partial rectification, production of dataset specific regex will be a lavishing choice for future studies. Computation of time and space complexity for lexical normalization will be attractive for strong theoretical justification. Combining strong models with clever normalization might be more promising and resource efficient. Lastly, the simplification of regex is on cards as they grow longer and deeper when more wrong spellings are added to lexicon. Trying a mix of general, specific and rule-based regex can augment the dataset with essential noise.

This dataset is going to become a benchmark for automatic regex generation for Roman Urdu text. In hope of its

rapid improvement over time, we shall make it public. We encourage everyone who utilizes it to contribute to it by:

1. Adding new words and their respective example strings, NL descriptions and regexes
2. Adding more examples strings (spell variations) for existing words
3. Updating the NL descriptions if needed
4. Updating the regexes accordingly

Author Contribution ST contributed to conceptualization, write-up and illustrations. TAR contributed to review, proof reading and copy editing.

Funding No funding.

Data Availability Data will be provided on demand.

Declaration

Competing interest The authors declare no competing interests.

References

1. Bartoli, A., De Lorenzo, A., Medvet, E., Tarlao, F.: Playing regex golf with genetic programming. In: Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation, pp. 1063–1070 (2014)
2. Bartoli, A., De Lorenzo, A., Medvet, E., Tarlao, F.: Inference of regular expressions for text extraction from examples. *IEEE Trans. Knowl. Data Eng.* **28**(5), 1217–1230 (2016)
3. Bartoli, A., De Lorenzo, A., Medvet, E., Tarlao, F.: Regex-based entity extraction with active learning and genetic programming. *ACM SIGAPP Appl. Comput. Rev.* **16**(2), 7–15 (2016)
4. Chen, Q., Banerjee, A., Demiralp, Ç., Durrett, G., Dillig, I.: Data extraction via semantic regular expression synthesis. *Proc. ACM Program. Lang.* **7**, 1848–1877 (2023)

5. Chen, Q., Wang, X., Ye, X., Durrett, G., Dillig, I.: Multi-modal synthesis of regular expressions. In: Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation, pp. 487–502 (2020)
6. Chida, N., Terauchi, T.: Repairing regular expressions for extraction. In: Proceedings of the ACM on Programming Languages, 7(PLDI), pp. 1633–1656 (2023)
7. Davis, J.C., Coghlan, C.A., Servant, F., Lee, D.: The impact of regular expression denial of service (ReDoS) in practice: an empirical study at the ecosystem scale. In: Proceedings of the 18th 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 246–256 (2018)
8. Gasparini, F., Rizzi, G., Saibene, A., Fersini, E.: Benchmark dataset of memes with text transcriptions for automatic detection of multi-modal misogynistic content. *Data Brief* **44**, 108526 (2022)
9. Gulwani, S., Marron, M.: Nlyze: Interactive programming by natural language for spreadsheet data analysis and manipulation. In: Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data, pp. 803–814 (2014)
10. Gvero, T., Kuncak, V.: Synthesizing Java expressions from free-form queries. In: Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications, pp. 416–432 (2015)
11. Jadidinejad, A.H., Sadr, H.: Improving weak queries using local cluster analysis as a preliminary framework. *Indian J. Sci. Technol.* **8**(5), 495–510 (2015)
12. Kalashami, M.P., Pedram, M.M., Sadr, H.: EEG feature extraction and data augmentation in emotion recognition. *Comput. Intell. Neurosci.* **2022**(1), 7028517 (2022)
13. Khodaverdian, Z., Sadr, H., Edalatpanah, S.A., Nazari, M.: An energy aware resource allocation based on combination of CNN and GRU for virtual machine selection. *Multimed Tools Appl* **83**(9), 25769–25796 (2024)
14. Kim, S.-H., Cheon, H., Han, Y.-S., Ko, S.-K.: SplitRegex: Faster Regex Synthesis via Neural Example Splitting (2021)
15. Kim, S.-H., Im, H., Ko, S.-K.: Efficient enumeration of regular expressions for faster regular expression synthesis. In: International Conference on Implementation and Application of Automata, pp. 65–76, Springer, Cham (2021)
16. Kim, S.-H., Cheon, H., Han, Y.-S., & Ko, S.-K.: Neuro-Symbolic Regex Synthesis Framework via Neural Example Splitting. *arXiv e-prints*. Retrieved from <https://arxiv.org/abs/2205.11258> (2022)
17. Kushman, N., Barzilay, R.: Using semantic unification to generate regular expressions from natural language (2013)
18. Lee, M., So, S., Oh, H.: Synthesizing regular expressions from examples for introductory automata assignments. In: Proceedings of the 2016 ACM SIGPLAN International Conference on Generative Programming: Concepts and Experiences, pp.70–80 (2016)
19. Li, Y., Li, S., Xu, Z., Cao, J., Chen, Z., Hu, Y., ... Cheung, S.-C.: TransRegex: multi-modal regular expression synthesis by generate-and-repair. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE), pp. 1210–1222, IEEE (2021)
20. Li, Y., Xu, Z., Cao, J., Chen, H., Ge, T., Cheung, S.-C., Zhao, H.: FlashRegex: deducing anti-ReDoS regexes from examples. In: 2020 35th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 659–671 (2020)
21. Lin, X.V., Wang, C., Zettlemoyer, L., Ernst, M.D.: NL2Bash: a corpus and semantic parser for natural language interface to the linux operating system. In: Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC 2018) (2018)
22. Locascio, N., Narasimhan, K., DeLeon, E., Kushman, N., Barzilay, R.: Neural generation of regular expressions from natural language with minimal domain knowledge. In: Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing, pp. 1918–1923 (2016)
23. Mahmood, Z., Safder, I., Nawab, R.M.A., Bukhari, F., Nawaz, R., Alfakeeh, A.S., Hassan, S.-U.: Deep sentiments in Roman Urdu text using Recurrent Convolutional Neural Network model. *Inform Process Manag* **57**(4), 102233 (2020)
24. McClurg, J., Claver, M., Garner, J., Vossen, J., Schmerge, J., Belviranli, M.E.: Optimizing regular expressions via rewrite-guided synthesis. In: Proceedings of the International Conference on Parallel Architectures and Compilation Techniques, pp. 426–438 (2022)
25. Mehmood, K., Essam, D., Shafi, K., Malik, M.K.: An unsupervised lexical normalization for Roman Hindi and Urdu sentiment analysis. *Inf. Process. Manage.* **57**(6), 102368 (2020)
26. Mohades Deilami, F., Sadr, H., Tarkhan, M.: Contextualized multidimensional personality recognition using combination of deep neural network and ensemble learning. *Neural Process. Lett.* **54**(5), 3811–3828 (2022)
27. Ouyang, L.: Bayesian Inference of Regular Expressions from Human-Generated Example Strings. *arXiv-1805*. Retrieved from <https://arxiv.org/abs/1805.00000> (2018)
28. Park, J.-U., Ko, S.-K., Cognetta, M., Han, Y.-S.: Softregex: Generating regex from natural language descriptions using softened regex equivalence. In: Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP), pp. 6425–6431 (2019)
29. Pertseva, E., Barbone, M., Rudek, J., Polikarpova, N.: Regex+: synthesizing regular expressions from positive examples. In: 11TH Workshop on Synthesis (2022)
30. Rahmani, K., Raza, M., Gulwani, S., Le, V., Morris, D., Radhakrishna, A., ... Tiwari, A.: Multi-modal program inference: a marriage of pre-trained language models and component-based synthesis. In: Proceedings of the ACM on Programming Languages, vol. 5, (OOPSLA), pp. 1–29 (2021)
31. Rana, T.A., Sahazadi, K., Rana, T., Arshad, A., Tubishat, M.: An unsupervised approach for sentiment analysis on social media short text classification in roman Urdu. *Trans Asian Low-Resour Lang Inform Process* **21**(2), 1–16 (2021)
32. Rebele, T., Tzompanaki, K., Suchanek, F.M.: Adding missing words to regular expressions. In: Proceedings on Advances in Knowledge Discovery and Data Mining: 22nd Pacific-Asia Conference, PAKDD 2018, Melbourne, VIC, Australia, June 3–6, 2018, Part II 22, pp. 67–79. Springer, New York (2018)
33. Redd, D., Gibson, B., Murtaugh, M.A., Goulet, J., Zeng-Treitler, Q.: Extract clinical measurement values using a regular expression pattern discovery algorithm vs support vector machine. In: E-Health 2018 Ict, Society And Human Beings (2018)
34. Sadr, H., Nazari Soleimandarabi, M.: ACNN-TL: attention-based convolutional neural network coupling with transfer learning and contextualized word representation for enhancing the performance of sentiment classification. *J. Supercomput.* **78**(7), 10149–10175 (2022)
35. Sadr, H., Soleimandarabi, M.N., Pedram, M., Teshnelab, M.: Unified topic-based semantic models: a study in computing the semantic relatedness of geographic terms. In: 5th International Conference on Web Research (ICWR), pp. 134–140. IEEE (2019)
36. Satti, F.A., Hussain, M., Ali, S.I., Saleem, M., Ali, H., Chung, T.C., Lee, S.: A semantic sequence similarity based approach for extracting medical entities from clinical conversations. *Inf. Process. Manage.* **60**(2), 103213 (2023)
37. Shen, Y., Jiang, Y., Xu, C., Yu, P., Ma, X., Lu, J.: ReScue: crafting regular expression DoS attacks. In: 33rd IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 225–235 (2018)

38. Uma, M., Sneha, V., Sneha, G., Bhuvana, J., Bharathi, B.: Formation of SQL from natural language query using NLP. In: International Conference on Computational Intelligence in Data Science (ICCIDS), pp. 1–5. IEEE (2019)
39. Vaithilingam, P., Pu, Y., Glassman, E.L.: The usability of pragmatic communication in regular expression synthesis. arXiv e-prints, arXiv-2308. <https://arxiv.org/abs/2308> (2023)
40. Wang, Y., Berant, J., Liang, P.: Building a semantic parser overnight. In: Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing, vol. 1: Long Papers, pp. 1332–1342 (2015)
41. Xu, L., Zhang, Y., Chinprutthiwong, P., Gu, G.: Automatic synthesis of network security services: a first step. In: 32nd International Conference on Computer Communications and Networks (ICCCN), pp. 1–10. IEEE (2023)
42. Ye, X., Chen, Q., Wang, X., Dillig, I., Durrett, G.: Sketch-driven regular expression generation from natural language and examples. *Trans. Assoc. Comput. Linguist.* **8**, 679–694 (2020)
43. Zhang, T., Lowmanstone, L., Wang, X., Glassman, E.L.: Interactive program synthesis by augmented examples. In: Proceedings of the 33rd Annual ACM Symposium on User Interface Software and Technology, pp. 627–648 (2020)
44. Zhong, Z., Guo, J., Yang, W., Peng, J., Xie, T., Lou, J.-G., ... Zhang, D. SemRegex: A semantics-based approach for generating regular expressions from natural language specifications. In: Proceedings of the 2018 conference on empirical methods in natural language processing (2018)
45. Zhong, Z., Guo, J., Yang, W., Xie, T., Lou, J.-G., Liu, T., Zhang, D. Generating regular expressions from natural language specifications: Are we there yet? In: Workshops at the Thirty-Second AAAI Conference on Artificial Intelligence (2018)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.